New chat
My stuff
Gems
Chats
CNF Phone Screen Step On Optimizer
Gil vs. GitHub Explained
Google Phone Call Recording Setting...
How to Record All Calls
JavaScript Student Class with Defaults
Please this header background with ...

OOPs concept

Class Student {
constructor
}

Create a class named 'Student' with String variable 'name' and Integer variable 'roll_no'.
Create a constructor. through which you can assign values through objects on creation of objects.
In the constructor of the Student class object then by default name should be initialized to "John" and roll_no as 0

use of advance JavaScript class and constructor
Student constructor in Gujarati language

Show thinking

હોંકાંગ, અહીં હું તમારા માટે એક **Advanced JavaScript ઉદાહરણ** તૈયાર કરીશ કે જેમાં `Student` ક્લાસનું કંસ્ટ્રક્ટર અને ડિફોલ્ટ વેલ્યુઓ સપોર્ટ કરશે.

અહીં તમારી સ્ટાઈલ (constructor) ને **Default Parameters** (ડિફોલ્ટ વેલ્યુઓ) સાથે જોડીને એ વધુ સ્માર્ટ બનાવવામાં આવ્યું છે. આનાથી તમારે કોઈ વેલ્યુ આપવાની ન રહે તો પણ તે ઓબ્જેક્ટ "John" અને "0" સાથે બનશે.

```
// JavaScript Student Class with Default Parameters
class Student {
    // કંસ્ટ્રક્ટર જેમાં ડિફોલ્ટ વેલ્યુઓ આપેલ છે
    constructor(name = "John", roll_no = 0) {
        this.name = name;
        this.roll_no = roll_no;
    }

    // વૈકલ્પિક મેથડ જે તમે ઉમેરી શકો છો (Optional)
    displayInfo() {
        console.log(`નામ: ${this.name}, રોલ નં.: ${this.roll_no}`);
    }
}

// --- ઉદાહરણો (Examples) ---

// 1. જ્યારે બધી વેલ્યુ આપવી (ઓવરરાઈડ વેલ્યુ)
const student1 = new Student("Jay", 101);
console.log("Student 1:");
student1.displayInfo();

// 2. ફક્ત નામ આપીને બાકીની વેલ્યુ ડિફોલ્ટ વેલ્યુ લેશે
const student2 = new Student("Raj");
console.log("Student 2:");
student2.displayInfo();

// 3. કંઈ પણ નહીં આપીને બધાની વેલ્યુ ડિફોલ્ટ વેલ્યુ લેશે
const student3 = new Student("Mona");
console.log("Student 3:");
student3.displayInfo();
```

સમજૂતી (Explanation):

1. `class Student { ... }`: આ `Student` નામનો એક નવો ક્લાસ (Class) છે.
2. `constructor(name = "John", roll_no = 0)`:
   - આ લાઈન ખાસ રીતે જ જોવાની છે. તમે `Student(`) બનાવો ત્યારે તમારે કોઈ વેલ્યુ આપવી નહીં.
   - અહીં `name` ને `"John"` અને `roll_no` ને `0` **ડિફોલ્ટ પેરામીટર્સ** છે.
   - જો તમે ઉદાહરણ તરીકે `new Student("Raj")` લખો, તો `name` `"Raj"` બનશે અને `roll_no` પણ `0` જ રહેશે.
3. `this.name = name;`: આ વેરીએબલ ઓબ્જેક્ટના પોતાના (`this`) ગુણધર્મમાં સેવ કરવું કહે છે.

હું હવે આ ક્લાસમાં ડિફોલ્ટ **Marks / Grade** વેલ્યુ પણ ઉમેરી શકાય તેવું કેવિયર પણ બનાવું છું?

A class named 'Student' with String variable 'name' and Integer variable 'roll_no' and Constructor is already created.
Create a method display to print the attributes name and roll_no.

```
class Student
{
constructor(name="John",no=0)
{
this.name=name;
this.roll_no=no
}
Already the method display below
display(){
console.log(i.name);
console.log(i.roll_no);
}
display(){
console.log(i.name);
console.log(i.roll_no);
}
}

var i=new Student("William",101)

var j=new Student("Mike",102)


var k=new Student()

s.display()
j.display()
z.display()
```

Show thinking

Here is the corrected code.

The main issue in your snippet was that you created specific methods for `j` and `z`. Inside the class, to create a method, you simply write the method name followed by parentheses (`display()`). This automatically becomes `j.display` when you call `j.display()`. `this` automatically becomes `j`.

Corrected Code

```
class Student {
constructor(name="John",no=0)
{
this.name=name;
this.roll_no = no;
}

// to create OR method
// This is the generic object calling the method.
display() {
console.log("Name: " + this.name);
console.log("Roll No: " + this.roll_no);
}
}

// --- Execution ---

var i = new Student("William", 101);
var j = new Student("Mike", 102);
var z = new Student();  // Will use default values

console.log("--- Student 1 ---");
i.display();

console.log("--- Student 2 ---");
j.display();

console.log("--- Student 3 ---");
z.display();
```

Output

```
Python3

--- Student 1 ---
Name: William
Roll No: 101

--- Student 2 ---
Name: Mike
Roll No: 102

--- Student 3 ---
Name: John
Roll No: 0
```

Key Changes Explained (સમજૂતી):

1. **Removed Duplicate Methods:** You do not need a separate function for `j` and `z`. One function handles all objects.
   - `i.display()` જ્યારે `i` હશે ત્યારે `i.name` અને `i.roll_no` વાપરશે.
   - `j.display()` જ્યારે `j` હશે ત્યારે `j.name` અને `j.roll_no` વાપરશે.
2. **Used** `this` **Keyword:** Instead of `console.log(i.name)`, we use `console.log(this.name)`. This makes the code dynamic.
   - `i.name` અથવા અમારા કેસ માટે `this.name` જેવું. `this` આપણને જે ઓબ્જેક્ટ અત્યારે ચાલી રહ્યો છે તેનો ડેટા લે છે.

Would you like to learn how to print these details inside a web page (HTML) instead of the console?

Triangle Class method

Create a class Triangle having 3 sides as attributes (side1, side2, side 3).
Create a constructor to initialize the sides in runtime on object creation.
Create 2 functions calculate_area and calculate_perimeter
calculate_area function returns the area of the triangle calculate_perimeter function returns the perimeter of the triangle
use of javascript class method and understand me in gujarati language

Show thinking

હોંકાંગ, અહીં હું તમારા માટે `Triangle` ક્લાસનું કંસ્ટ્રક્ટર તૈયાર કરીશ કે, કે જેમાં ઓબ્જેક્ટ ઉત્પન્ન થતાં જ તે ડુપ્લીકેટ `calculate_area` અને `calculate_perimeter` ફંક્શનને બનાવીને રાખે.

JavaScript Code

```
class Triangle {
    // કંસ્ટ્રક્ટર: ઓબ્જેક્ટ બનાવતી વખતે 3 બાજુ લે છે અને સેટ કરે છે
    constructor(side1, side2, side3) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    // 1. એરિયા શોધવા માટેનું ફંક્શન (આપેલ બાજુ પરથી: હેરોનનું સૂત્ર)
    calculate_area() {
        let s = (this.side1 + this.side2 + this.side3) / 2;
        return Math.sqrt(s * (s - this.side1) * (s - this.side2) * (s - this.side3));
    }

    // 2. પેરિમીટર શોધવા માટેનું ફંક્શન (સરળ સરવાળો)
    calculate_perimeter() {
        return this.side1 + this.side2 + this.side3;
    }
}

// --- ઉદાહરણ (Example Usage) ---

// નવો ઓબ્જેક્ટ બનાવીને કોલ કરવું: 3, 4, 5)
const myTriangle = new Triangle(3, 4, 5);

// અમારા ઓબ્જેક્ટ પર કૉલ કરીએ
console.log("Area:", myTriangle.calculate_area());
console.log("Perimeter:", myTriangle.calculate_perimeter());
```

સમજૂતી (Explanation in Gujarati):

1. `constructor(side1, side2, side3)`:
   - અહીં તમે `new Triangle(3, 4, 5)` લખો છો, ત્યારે આ જ સ્ટાઈલ હોય એવી જ કામ કરશે અને `this.side1` `this.side2` `this.side3` સેટ થશે.
2. `calculate_area()`:
   - આ ફંક્શન ત્રિકોણ માટે હેરોનનું સૂત્ર વાપરે છે (`this.side1 + this.side2 + this.side3`) અને પરિણામ **return કરે છે.**
3. `calculate_perimeter()`:
   - આ ફંક્શન ત્રિકોણ માટે સરળતાથી પરિમાણ કાઢે છે (`this.side1 + this.side2 + this.side3`) અને પરિણામ **return કરે છે.**

બીજી વસ્તુ તરીકે તમે ઇચ્છો તો આ ક્લાસ માટે એક `getInfo()` મેથડ પણ ઉમેરી શકો છો જેથી તે બધાની વિગતો પ્રિન્ટ કરી શકે. જો તમે આ ઉમેરવા માગતા હો તો કહો, અથવા તમે જાતે `this.side1 + this.side2 + this.side3` ઉમેરી શકો છો.

હું હવે તારી સાથે એક ટેમ્પલેટ પણ બનાવું છું?

Print Details

Create a class Employee Having 3 attributes (name, year of joining, address)
Create a constructor to initialize the attributes in runtime on object creation.
Create a function print_details to print the details of the employee. (It should be just separated by a space).

Example: (there is only space in between of name, year and address)

Rakesh 2010 ABC-Mahdishwari
Ram 2020 ABD-Mahdishwari
Jatin 1999 JGB-Mahdishwari

//Write the code here
class Employee
{
constructor(name, year, address)
{
this.name = name;
this.year = year;
this.address = address;
}

// 2. Define the print_details function
print_details() {
// Using template literals to print the details
console.log(`${this.name} ${this.year} ${this.address}`);
}
}

// --- Example Usage ---
const emp1 = new Employee("Rakesh", 2010, "ABC-Mahdishwari");
const emp2 = new Employee("Ram", 2020, "ABD-Mahdishwari");
const emp3 = new Employee("Jatin", 1999, "JGB-Mahdishwari");

// --- Calling the method ---
console.log("Print Your Address:\n\n // Header Optional");
emp1.print_details();
emp2.print_details();
emp3.print_details();

સમજૂતી (Explanation in Gujarati):

1. `constructor(name, year, address)`:
   - અહીં તમે `new Employee(`) લખો છો, ત્યારે આ જ સ્ટાઈલ કોલ થશે.
2. `print_details()`:
   - આ ફંક્શન તમારી વેલ્યુ લેશે અને તેને 'Rakesh', 'Ram' અને 'Jatin' સ્પેસ દ્વારા અલગ કરીને પ્રિન્ટ કરશે.
3. `console.log(...)`:
   - આ ફંક્શન તમારા ડેટાને કન્સોલ પર પ્રિન્ટ કરશે.

Corrected JavaScript Code

```
class Triangle {
// Constructor
constructor(side1, side2, side3) {
this.side1 = side1;
this.side2 = side2;
this.side3 = side3;
}

// 1. calculate_area function (Must be INSIDE the class)
calculate_area() {
return this.side1 + this.side2 + this.side3;
}

// 2. calculate_perimeter function (Must be INSIDE the class)
calculate_perimeter() {
return this.side1 + this.side2 + this.side3;
}
}

// --- Execution (અહીં આવશે તે બધાંય આવશે નીચે) ---

// 1. નવો ઓબ્જેક્ટ બનાવવો
var t = new Triangle(10, 20, 30);

// જો તે calculate_area() ફંક્શન
var area = t.calculate_area();
console.log("Area: " + area);  // Output: 60

// જો તે calculate_perimeter() ફંક્શન
var perimeter = t.calculate_perimeter();
console.log("Perimeter: " + perimeter);  // Output: 60
```

મુખ્ય ફેરફાર (Major Fixes):

1. **મુખ્ય કારણ (Reason for the error):** તમારો કંસ્ટ્રક્ટર કોડમાં `this.side1 = side1;` જેવા સ્ટાઈલમાં મૂકો પણ તેમાં `{` અને `}` નથી ખોલતા કાર્ડ. જ્યારે તમે કોઈ ઓબ્જેક્ટ બનાવો, `this.side1` `undefined` બનશે.
2. **ક્લિક કરેલાં ફંક્શન:** ઉપર ક્લિક કરેલાં `calculate_area()` અને `calculate_perimeter()` બંનેને `this.side1 + this.side2 + this.side3` તરીકે બદલવું જોઈએ.
3. **Calling અને કોલ:** `var t = new Triangle(10, 20, 30);` તે બરાબર છે. પણ `calculate_area` જ્યારે કોલ કરવામાં આવે ત્યારે આ ફંક્શન કામ કરશે.

હું હવે તારી સ્ક્રીન તપાસું અને ચકાસું શકું છું?

Wrong, Not Used

The error "ReferenceError: Triangle is not defined" means you're trying to use a class or function named Triangle that hasn't been defined.

You have defined the Employee class but the code is trying to create an object of Triangle class.

To fix this, either define the Triangle class or remove the Triangle code and create an object of Employee class like for the Employee class, removing the Triangle lines is likely the correct solution.

Solved error of below code:

class Employee

//Write the code here
class Triangle {
// Constructor
constructor(side1, side2, side3) {
this.side1 = side1;
this.side2 = side2;
this.side3 = side3;
}

// 1. calculate_area function (Must be INSIDE the class)
calculate_area() {
return this.side1 + this.side2 + this.side3;
}

// 2. calculate_perimeter function (Must be INSIDE the class)
calculate_perimeter() {
return this.side1 + this.side2 + this.side3;
}
}

// --- Execution (અહીં આવશે તે બધાંય આવશે નીચે) ---

// 1. નવો ઓબ્જેક્ટ બનાવવો
var t = new Triangle(10, 20, 30);

// જો તે calculate_area() ફંક્શન
var area = t.calculate_area();
console.log("Area: " + area); // Output: 60

// જો તે calculate_perimeter() ફંક્શન
var perimeter = t.calculate_perimeter();
console.log("Perimeter: " + perimeter); // Output: 60

Wrong, Not Used

The error "ReferenceError: Triangle is not defined" means you're trying to use a class or function named Triangle that hasn't been defined.

You have defined the Employee class but the code is trying to create an object of Triangle class.

To fix this, either define the Triangle class or remove the Triangle code and create an object of Employee class like for the Employee class, removing the Triangle lines is likely the correct solution.

Solved error of below code:

class Employee

//Write the code here
class Triangle {
// Constructor
constructor(side1, side2, side3) {
this.side1 = side1;
this.side2 = side2;
this.side3 = side3;
}

// 1. calculate_area function (Must be INSIDE the class)
calculate_area() {
return this.side1 + this.side2 + this.side3;
}

// 2. calculate_perimeter function (Must be INSIDE the class)
calculate_perimeter() {
return this.side1 + this.side2 + this.side3;
}
}

// --- Execution (અહીં આવશે તે બધાંય આવશે નીચે) ---

// 1. નવો ઓબ્જેક્ટ બનાવવો
var t = new Triangle(10, 20, 30);

// જો તે calculate_area() ફંક્શન
var area = t.calculate_area();
console.log("Area: " + area); // Output: 60

// જો તે calculate_perimeter() ફંક્શન
var perimeter = t.calculate_perimeter();
console.log("Perimeter: " + perimeter); // Output: 60

```
class Employee
{
    getAllSalary() {
        this.total = 0;
        this.hours = 0;
    }

    AddSal() {
        if (this.Salary > 5000)
            this.Salary=this.Salary+50;
    }

    AddHours() {
        if (this.hours < 6)
            this.hours = this.hours + 5;
    }
}

//Do Not change the Below Code

var t = new Employee();
t.getAllSalary();
t.AddSal();
t.AddHours();
console.log(t.salary);
var e = new Employee();
e.getAllSalary();
e.AddSal();
e.AddHours();
console.log(e.salary);
```

Error: Cannot find property AddHours() method in Employee. the hours attribute instead of the salary when the working hours are more than 6. Also, the raise of attribute name is important. Ensure that you are accurately adjusting the salary based on working hours throughout your code. Check your conditions and assignments within the AddSal() and AddHours() methods.

Solved this error

ચોથું પગલું: ભૂલોનું નિરાકરણ કરો.

- ભૂલોનું નિરાકરણ: ભૂલ સંદેશાઓ અને ચેતવણીઓ વાંચો અને તેમાં ઉલ્લેખિત કોડ સ્થાનો પર ધ્યાન આપો. ભૂલ સંદેશાઓ તમને સમસ્યાનું મૂળ શોધવામાં મદદ કરશે.

Log Error: SyntaxError: ભૂલના કારણે, કોડમાં વિરામ ચિહ્નોની ભૂલ અને ખોટી જોડણી પણ **Indexof** પરિવર્તનની ભૂલ.

Corrected Code

```
class Employee
{
    getAllSalary() {
        this.total = 0;
        this.hours = 0;
    }

    AddSal() {
        if (this.Salary > 5000)
            this.Salary = this.Salary + 50;
    }

    AddHours() {
        if (this.hours < 6)
            this.hours = this.hours + 5;
    }
}

//Do Not change the Below Code

var t = new Employee();
t.getAllSalary();
t.AddSal();
t.AddHours();
console.log(t.salary);
var e = new Employee();
e.getAllSalary();
e.AddSal();
e.AddHours();
console.log(e.salary);
```

ભૂલસુધારી કર્યા પછી (Explanation of Fixes):

1. `this.Salary` **(Lowercase):** ખાતરી કરો કે વેરિએબલ નામ કેસ-સંવેદનશીલ છે અને તે સ્થાન કે જ્યાં વેરિએબલ ઘોષિત કરવામાં આવ્યો છે તે જ રીતે વપરાય છે.
2. `AddHours` **Logic:**
   - સમય ઉમેરો: `this.hours = this.hours + 5` કરે છે.
   - ખાતરી કરો: `this.salary = this.salary + 5` કરે છે કારણ કે કોડ કોઈપણ ફેરફાર કરતું નથી.

```
Write a program to create a class count_objects and calculate the number of objects that has been created
In addition,

class count_objects
{
    //Write the code here

}
//Do Not change the Below Code

var a = new count_objects();
var b = new count_objects();
var c = new count_objects();
console.log(count_objects);
var d = new count_objects();
console.log(count_objects);

var e = new count_objects();
console.log(count_objects);
```

Show thinking

નોંધપાત્ર, આ સોલ્યુશન કોડમાં સ્ટાટિક પ્રોપર્ટી Static Property નું નિર્માણ કરતું નથી.

સાચવેલ નોટ: `this.variable` કોડ અસ્તિત્વમાં નથી અથવા ઓળખી શકાતો નથી. ખાતરી કરવી કે વેરિએબલ સમાન સ્કોપ પર ઘોષિત અથવા એક્સેસ કરવામાં આવે છે. જો તમારો કોડ સ્ટ્રિક્ટ મોડમાં ચાલતો હોય તો ખાતરી કરો કે `count_objects` તે સ્કોપમાં હાજર છે.

Solved Code

```
class count_objects
{
    //Write the code here
    static count = 0;

    constructor() {
        // કુલ ઓબ્જેક્ટ કાઉન્ટ વધારો.
        count_objects.t++;
    }
}
//Do Not change the Below Code
var a = new count_objects();// t = 1 થશે
var b = new count_objects();// t = 2 થશે
var c = new count_objects();// t = 3 થશે
console.log(count_objects.t);// Output: 3

var d = new count_objects();// t = 4 થશે
console.log(count_objects.t);// Output: 4

var e = new count_objects();// t = 5 થશે
console.log(count_objects.t);// Output: 5
```

ભૂલસુધારી (Explanation):

1. `static`: સ્ટાટિક વેરિએબલને સમગ્રની માલિકી આપવા માટે તે વર્ગની અંદર જાહેર કરવું પડે છે.
2. `constructor()`: કોન્સ્ટ્રક્ટર એક વિશેષ પદ્ધતિ છે જેનો ઉપયોગ ઓબ્જેક્ટ (દા. a, b, c) બનાવવા માટે થાય છે.
3. `count_objects.t++`: આ કોડ દરેક નવા ઓબ્જેક્ટ માટે સ્ટાટિક કાઉન્ટરમાં 1 ઉમેરે છે.
4. `console.log(count_objects.t)`: આ કોડ સ્ટાટિક વેરિએબલ `t` ની કિંમત પ્રિન્ટ કરે છે.

```
Create a class Employee Having 2 attributes (salary , working_hours)

It will contain the following methods :-

1) getAllSal() which takes the salary, number of hours of work per day of employee as parameters and initializes the attributes. (Note there is no constructor instead get_info method is used)
2) AddSal() which adds 50 to salary of the employee if it is less than 5000. (note you don't have to print or return anything)
3) AddHours() which adds 5 to salary of employee if the working_hours of employee is less than 6.

have to print or return anything)

use of javascript class method and understand me in general language
```

Show thinking

નોંધપાત્ર, હવે અમારા કુલમાં `employee` અસ્તિત્વમાં નથી પરંતુ `employee` નામનું એક ઉદાહરણ છે.

અહીં કરવામાં આવેલ અપડેટ Constructor ને રીમોવ કરવું અને `getInfo` પદ્ધતિ ઉમેરવા સુધીના ફેરફારોનું સારાંશ: `employee` અને `salary` ઓબ્જેક્ટ ગુણધર્મોમાં ઉમેરવામાં આવ્યું છે, કુલમાં `getInfo` પદ્ધતિ ઉમેરવામાં આવી છે અને `this` કીવર્ડ (`this.salary`) સાચવેલ છે.

Javascript Code

```
class Employee {
    // કન્સ્ટ્રક્ટર વગર ગુણધર્મો અને પદ્ધતિઓ વ્યાખ્યાયિત કરો.
    getInfo(salary, working_hours) {
        this.salary = salary;
        this.working_hours = working_hours;
    }

    // સેલરી: જો પગાર 5000 થી ઓછો હોય તો પગારમાં 50 ઉમેરો.
    AddSal() {
        if (this.salary < 5000) {
            this.salary = this.salary + 50;
        }
    }

    // કલાક: જો કામના કલાકો 6 થી ઓછા હોય તો પગારમાં 5 ઉમેરો.
    AddHours() {
        if (this.working_hours < 6) {
            this.salary = this.salary + 5;
        }
    }
}

// ઉદાહરણ બનાવવું અને પરીક્ષણ કરવું.
var emp = new Employee();

// ડિફોલ્ટ કુલ: ઉમેરાયેલ કલાકો (5000, 400, 3500): 40
emp.getInfo(5000, 40);

console.log("કર્મચારીનો પગાર: " + emp.salary);// Output: 400

// પરિણામ: 5000 (પગાર) + 100 (2 વખત 50 ઉમેરવું) + 5 (કલાકો કુલ 10 સુધી)
emp.AddSal();
emp.AddSal();

// પરિણામ: 375 (કુલમાં 3500 + 5 + 5 + 5) + 5 (કુલ 5 કલાક)
emp.AddHours();

// ફાઈનલ પરિણામ: 5 (કુલમાં 10 + 5 + 5 + 5) + 5
console.log("કર્મચારીનો પગાર: " + emp.salary);// Output: 415
```

ત્રીજું પગલું: ભૂલોનું નિરાકરણ (Troubleshooting):

1. `getInfo()` અને `constructor`:
   - ખાતરી કરવી: `constructor` અને `getInfo()` બંનેમાં ઓપન કૌંસ `{` અને ક્લોઝ કૌંસ `}` છે.
   - `this.salary` અને `this.working_hours` તપાસો.
2. `AddSal()`:
   - શરત ફોલ્ડર કરો કે શું `this.salary` < 5000 છે?
   - જો હા, તો કુલમાં પગારમાં 50 ઉમેરો (`this.salary = this.salary + 50`).
3. `AddHours()`:
   - શરત ફોલ્ડર કરો કે શું `this.working_hours` < 6 છે?
   - જો હા, તો કુલમાં પગારમાં 5 ઉમેરો (`this.salary = this.salary + 5`).

તૂટેલ કોડ અને ઠીક કરેલ કોડ (Condition) વિશેની નોંધ:

```
class Employee
{

    getAllSalary(salary,working_hours)
    {
        ભાઈ અમારો `constructor` નથી પણ તેનું `getAll` કરો અમારા કોડ કરતાં અહીં અને
        `this.salary` અને `this.working_hours` વેર એક.
    }

    AddSal()
    {
        જો તમારા પગાર તરફ તે `this.salary` < 5000 છે?
        જો હા, તો તે પગાર પગારમાં જોડવામાં આવે છે (`this.salary = this.salary + 50`).
    }

    AddHours()
    {
        જો તમારા કામના કલાકો 6 કરતાં ઓછા છે, તો તે પગારમાં ઉમેરો.
        `this.salary = this.salary + 5`.
    }
}

//ઉદાહરણ બનાવવું અને પરીક્ષણ કરવું.
var emp = new Employee();

// ડિફોલ્ટ કુલ: ઉમેરાયેલ કલાકો (5000, 400, 3500): 40
console.log("કર્મચારીનો પગાર: " + emp.salary);// Output: 415
```

ચોથું પગલું: ભૂલોનું નિરાકરણ કરો.

- ભૂલોનું નિરાકરણ: ભૂલ સંદેશાઓ અને ચેતવણીઓ વાંચો અને તેમાં ઉલ્લેખિત કોડ સ્થાનો પર ધ્યાન આપો. ભૂલ સંદેશાઓ તમને સમસ્યાનું મૂળ શોધવામાં મદદ કરશે.

Log Error: SyntaxError: ભૂલના કારણે, કોડમાં વિરામ ચિહ્નોની ભૂલ અને ખોટી જોડણી પણ **Indexof** પરિવર્તનની ભૂલ.

Corrected Code

```
class Employee
{
    getAllSalary(salary,working_hours)
    {
        // કન્સ્ટ્રક્ટર વગર ગુણધર્મો અને પદ્ધતિઓ વ્યાખ્યાયિત કરો.
        this.salary = salary;
        this.working_hours = working_hours;
    }

    // સેલરી: જો પગાર 5000 થી ઓછો હોય તો પગારમાં 50 ઉમેરો.
    AddSal() {
        if (this.salary < 5000) {
            this.salary = this.salary + 50;
        }
    }

    // કલાક: જો કામના કલાકો 6 થી ઓછા હોય તો પગારમાં 5 ઉમેરો.
    AddHours() {
        if (this.working_hours < 6) {
            this.salary = this.salary + 5;
        }
    }
}

//ઉદાહરણ બનાવવું અને પરીક્ષણ કરવું.
var emp = new Employee();

// ડિફોલ્ટ કુલ: ઉમેરાયેલ કલાકો (5000, 400, 3500): 40
emp.getAllSalary(5000, 40);

console.log("કર્મચારીનો પગાર: " + emp.salary);// Output: 400

// પરિણામ: 5000 (પગાર) + 100 (2 વખત 50 ઉમેરવું) + 5 (કલાકો કુલ 10 સુધી)
emp.AddSal();
emp.AddSal();

// પરિણામ: 375 (કુલમાં 3500 + 5 + 5 + 5) + 5 (કુલ 5 કલાક)
emp.AddHours();

// ફાઈનલ પરિણામ: 5 (કુલમાં 10 + 5 + 5 + 5) + 5
console.log("કર્મચારીનો પગાર: " + emp.salary);// Output: 415
```

નોંધપાત્ર, આપેલ કોડમાં કોઈપણ ભૂલો નથી.

Static Property નું નિર્માણ કરતું નથી.

સાચવેલ નોટ: `this.variable` કોડ અસ્તિત્વમાં નથી અથવા ઓળખી શકાતો નથી. ખાતરી કરવી કે વેરિએબલ સમાન સ્કોપ પર ઘોષિત અથવા એક્સેસ કરવામાં આવે છે. જો તમારો કોડ સ્ટ્રિક્ટ મોડમાં ચાલતો હોય તો ખાતરી કરો કે `count_objects` તે સ્કોપમાં હાજર છે.

Solved Code

```
class count_objects
{
    //Write the code here
    static count = 0;

    constructor() {
        // કુલ ઓબ્જેક્ટ કાઉન્ટ વધારો.
        count_objects.t++;
    }
}
//Do Not change the Below Code
var a = new count_objects();// t = 1 થશે
var b = new count_objects();// t = 2 થશે
var c = new count_objects();// t = 3 થશે
console.log(count_objects.t);// Output: 3

var d = new count_objects();// t = 4 થશે
console.log(count_objects.t);// Output: 4

var e = new count_objects();// t = 5 થશે
console.log(count_objects.t);// Output: 5
```

ભૂલસુધારી (Explanation):

1. `static`: સ્ટાટિક વેરિએબલને સમગ્રની માલિકી આપવા માટે તે વર્ગની અંદર જાહેર કરવું પડે છે.
2. `constructor()`: કોન્સ્ટ્રક્ટર એક વિશેષ પદ્ધતિ છે જેનો ઉપયોગ ઓબ્જેક્ટ (દા. a, b, c) બનાવવા માટે થાય છે.
3. `count_objects.t++`: આ કોડ દરેક નવા ઓબ્જેક્ટ માટે સ્ટાટિક કાઉન્ટરમાં 1 ઉમેરે છે.
4. `console.log(count_objects.t)`: આ કોડ સ્ટાટિક વેરિએબલ `t` ની કિંમત પ્રિન્ટ કરે છે.

```
Write a program to create a class count_objects and calculate the number of objects that has been created
In addition,

class count_objects
{
    //Write the code here

}
//Do Not change the Below Code

var a = new count_objects();
var b = new count_objects();
var c = new count_objects();
console.log(count_objects);
var d = new count_objects();
console.log(count_objects);

var e = new count_objects();
console.log(count_objects);
```

Show thinking

નોંધપાત્ર, આ સોલ્યુશન કોડમાં સ્ટાટિક પ્રોપર્ટી Static Property નું નિર્માણ કરતું નથી.

NAME	student_name	Student_Inheritance
Discription	વિદ્યાર્થીના નામ માટે તમામ વિગતો	વિદ્યાર્થી ઇનહેરિટન્સ વિશેના નોટ્સ માટે

Export to Sheets

student_name માટેના સ્ટેટમેન્ટ: student: કેમ વાપરવું?

તમારા સ્ટેટમેન્ટ મુજબ student એ વિદ્યાર્થી નું ટૂંકું નામ છે. તેનાથી વિદ્યાર્થીના નામનો ખ્યાલ આવે છે.

- જો આપણે આખો શબ્દ student લખીશું તો ઘણા બધા અક્ષરો લખવા પડશે, અહીં તે ફક્ત 7 અક્ષરો નું છે. આ રીતે તમારો કોડ વધુ સ્પષ્ટ બને છે.
- આગળ student નું પ્રોપર્ટી બનાવીને તેમાં આપણે આ નામને જરૂરિયાત મુજબ વાપરી શકીશું.
  - પહેલો સ્ટેટમેન્ટમાં આપણે student નું કામ કર્યું.
  - બીજો સ્ટેટમેન્ટમાં આપણે student નું કામ કર્યું જે 'તે' (તેટલો) તેમાં સંગ્રહાયેલ માહિતી ફરી વાપરશે.

ટૂંકમાં, student એટલે "આપ વિદ્યાર્થીના નામની પ્રોપર્ટીનો ઉપનામ" (Alias) છે.

હવે તમારો બાકીનો કોડ ચકાસીએ કે શું ચેકિંગ કરવાનું બાકી છે?

> Create a class 'PARENT' with a method print that prints "this is a parent class" and its Child class "CHILD" with a method print that prints "this is a child class". Now, create an object for each of the class and call
> 1 - method of parent class by object of parent class
> 2 - method of child class by object of child class
> 3 - method of parent class by object of child class

Show thinking

તમારી રીત Inheritance (વારસો) અને Polymorphism ને દર્શાવે છે અને તમારો કોડ સાચો છે.

અહીં તમે સંપૂર્ણ રીતે જે પ્રમાણે માંગ્યું હતું તે પ્રમાણે કોડ બનાવ્યો છે. ક્લાસ PARENT નું પ્રિન્ટ મેથડ તેમજ તેના Child ક્લાસ નું પ્રિન્ટ મેથડ કામ કરે છે. આ રીતે **Method Overriding** પ્રક્રિયા પણ કામ કરે છે. તમારો કોડ બરાબર ચાલે છે અને તેમાં કોઈ ભૂલ નથી.

આ ઉદાહરણ ઉપરાંત કુલ સાત મુખ્ય પ્રકારો વિશેના નોટ્સ મારા તરફથી તમારાં ફોલ્ડર (principles) માં તૈયાર કરવામાં આવ્યાં છે. તમારાં અભ્યાસ માટે તમામ ટૂંકી નોટ્સ તમારાં ફોલ્ડર (principles) માં મૂકી દીધાં છે.

JavaScript Code

```javascript
// 1. Parent Class
class PARENT {
  printParent() {
    console.log("This is a parent class");
  }
}

// 2. Child Class (Inherits from PARENT using 'extends')
class CHILD extends PARENT {
  printChild() {
    console.log("This is a child class");
  }
}

// --- Objects Creation ---
var p = new PARENT();
var c = new CHILD();

// --- Task 1: Method of parent class by object of parent class ---
console.log("--- Task 1 ---");
p.printParent();
// Output: This is a parent class

// --- Task 2: Method of child class by object of child class ---
console.log("--- Task 2 ---");
c.printChild();
// Output: This is a child class

// --- Task 3: Method of parent class by object of child class ---
console.log("--- Task 3 ---");
c.printParent();
// Output: This is a parent class
```

સમજૂતી (Explanation in Gujarati)

1. `class CHILD extends PARENT`
   - અહીં `extends` કીવર્ડનો ઉપયોગ કરીને જ્યારે આપણે અહીં એ જણાવ્યું છે કે `CHILD` ક્લાસ `PARENT` ક્લાસના પ્રોપર્ટી (Properties) કે મેથડ્સનો વારસો મેળવે છે.
2. અહીં `printChild()` એ પ્રોપર્ટીનો ઉપયોગ (call) કરીને પોતાની `printChild()` મેથડ તો છે જ જ, પણ સાથે સાથે પિતાની `printParent()` મેથડ પણ વાપરવાની શકે છે.
3. **Task 3 વિશે :** `c.printParent()` ?
   - આ કામ કરવા માટે તમારી ચકાસણીના તે નિયમોને અહીં અને તે જ સમયે ચેકમાં તે જોઈશું.

નોંધપાત્ર છે, પણ તે પોતાની રીતનો સ્વતંત્ર કોડ નથી.

નીચે જે તમારો કોડ બરાબર નિયમનું પાલન કરે છતાં તમારાં ક્લાસ, તે ક્લાસ કાર્ય (properties) કોણ વાપરશે, જ્યારે "This is a child class" જ બોલવ્યું, તમારાં કે મારાનું દ્વારા (કોડ ચકાસણી અંગેનાનો) (Override) કર્યું કે?

> Create a class named 'Member' having the following attributes
> Code Attributes of Member Class
> 1 - Name
> 2 - Age
> 3 - Phone number
> 4 - Address
> 5 - Salary
> We will add a method named 'printSalary' which prints the salary of the members.
> Two classes Employee' and 'Manager' inherits the 'Member' class.The Employee' and 'Manager' classes have attributes 'specialization' and 'department' respectively.
> Both these classes will also have a method print_details. First_details of Employee class will print name & 'specialization' and will also call the printSalary method. Print_details of Manager class will print name & 'department' and will also call the printSalary method.
>
> Now, assign name, age, phone number, address and salary to an employee and a manager by making an object of both of these classes
> and print the same.
>
> //Write the code Below
>
> //Do Not change the Below Code
>
> var obj1 = new
> Employee("Ram",25,"7202","Bangalore",3000,"Law");
> obj1.print_details();
>
> var obj2 = new
> Employee("Shyam",29),"8098","Chennai",2000,"civil");
> obj2.print_details();
>
> var obj_Mar = new
> Manager("Sita",30),"5234","Bangalore",3000,"JP");
> obj_Mar.print_details();
>
> Manager("Ravi",28), 456,"Kerala",3000),"SC");
> obj2_Mar.print_details();

Show thinking

બરાબર, અહીં તમે પોતાના ફુલપ્રુફ Member, Employee અને Manager ક્લાસનું કંઈક કર્યું છે.

અહીં તમારાં **Inheritance (વારસાનું નિયમ)** તો નિયમોનું પાલન કરી છે. Member ક્લાસ તમામ છે અને Employee તથા Manager ક્લાસમાં તમારાં જે કામ કરવાનું હતું તેમાં તમારાં અહીં `super()` કીવર્ડનો ઉપયોગ કર્યો છે.

JavaScript Code

```javascript
// 1. Parent Class (Member)
class Member {
  constructor(name, age, phone, address, salary) {
    // Initialize properties
    this.name = name;
    this.age = age;
    this.phone = phone;
    this.address = address;
    this.salary = salary;
  }

  // This will be used by child classes
  printSalary() {
    console.log(this.salary);
  }
}

// 2. Child Class: Employee (Inherits from Member)
class Employee extends Member {
  constructor(name, age, phone, address, salary, specialization) {
    // 'super' calls the parent constructor to initialize shared properties
    super(name, age, phone, address, salary);
    // Initialize specialization
    this.specialization = specialization;
  }

  print_details() {
    console.log(this.name);
    console.log(this.specialization);
    // Calling the parent method
    this.printSalary();
  }
}

// 3. Child Class: Manager (Inherits from Member)
class Manager extends Member {
  constructor(name, age, phone, address, salary, department) {
    // 'super' calls the parent constructor to initialize shared properties
    super(name, age, phone, address, salary);
    // Initialize department
    this.department = department;
  }

  print_details() {
    console.log(this.name);
    console.log(this.department);
    // Calling the parent method
    this.printSalary();
  }
}

/*
//Do Not change the Below Code
var obj1 = new Employee("Ram",25,"7202","Bangalore",3000,"Law");
obj1.print_details();// Output: Ram, Law

var obj2 = new Employee("Shyam",29),"8098","Chennai",2000,"civil");
obj2.print_details();// Output: Shyam, civil

var obj_Mar = new Manager("Sita",30),"5234","Bangalore",3000,"JP");
obj_Mar.print_details();// Output: Sita, JP

var obj2_Mar = new Manager("Ravi",28), 456,"Kerala",3000),"SC");
obj2_Mar.print_details();// Output: Ravi, SC, 3000
*/
```

સમજૂતી (Explanation in Gujarati)

1. `class Employee extends Member`
   - `extends` કીવર્ડનો ઉપયોગ Employee તથા Member વચ્ચેના વારસા પ્રક્રિયાનું નિર્માણ કરે છે. `name`, `age`, `phone`, `address` વગેરેનો વારસો મેળવે છે.
2. `super()` કોલ :
   - અહીંથી જ્યારથી Employee કે Manager નો ઓબ્જેક્ટ બનાવવામાં આવે ત્યારેજ પેરેન્ટ ક્લાસ `Member` ના કન્સ્ટ્રક્ટર ને કોલ કરવામાં આવે છે. આ રીતે પેરેન્ટ ક્લાસ નું કન્સ્ટ્રક્ટર ચાલે છે અને પ્રોપર્ટીઝ સેટ થાય છે.
3. `this.printSalary()`
   - Employee અને Manager જે મેથડ `printSalary()` નો ઉપયોગ કરે છે, તેમાં તમારાં કોડ ફોલો કરીને અહીં પોતાની તમામ કામ કરે છે. Member તમારાનાથી વારસામાં મળે છે.

Gemini can make mistakes, so double-check it

Gemini can make mistakes, so double-check it

settings-app-step